



**YENEPOYA**  
Deemed to be University

**YENEPOYA**  
**(DEEMED TO BE UNIVERSITY)**

**Project – High Level Design**

**on**

**Edge AI for Manufacturing Real-time  
Monitoring**

**Student Name: MUHAMMED SUBAIR CH**

**Industry Mentor: Mr. Sumit K Shukla**

**Guided by: Mr Praveen**

## **Table of Contents**

|                                      |    |
|--------------------------------------|----|
| 1. Introduction. ....                | 6  |
| 1.1. Scope of the document. ....     | 6  |
| 1.2. Intended Audience ....          | 6  |
| 1.3. System overview. ....           | 6  |
| 2. System Design. ....               | 6  |
| 2.1. Application Design ....         | 6  |
| 2.2. Process Flow. ....              | 6  |
| 2.3. Information Flow. ....          | 7  |
| 2.4. Components Design ....          | 8  |
| 2.5. Key Design Considerations ....  | 8  |
| 2.6. API Catalogue. ....             | 8  |
| 3. Data Design. ....                 | 8  |
| 3.1. Data Model ....                 | 8  |
| 3.2. Data Access Mechanism ....      |    |
| 3.3. Data Retention Policies.....    | 9  |
| 3.4. Data Migration ....             | 9  |
| 4. Interfaces ....                   | 9  |
| 5. State and Session Management .... | 10 |
| 6. Caching ....                      | 10 |
| 7. Non-Functional Requirements ....  | 10 |
| 7.1. Security Aspects ....           | 10 |
| 7.2. Performance Aspects ....        | 10 |
| 8. References ....                   | 11 |

# 1. Introduction

The integration of Artificial Intelligence with edge computing has enabled intelligent systems to operate directly on local devices without relying on cloud infrastructure. This paradigm, known as **Edge AI**, ensures faster processing, reduced latency, and enhanced data privacy—critical factors in healthcare environments.

The **AI-Based Edge Object Detection for Healthcare System** is designed to perform realtime image classification using a lightweight **MobileNet TensorFlow Lite (TFLite)** model. The system captures live video input through a webcam, processes the frames locally, and predicts objects without requiring internet connectivity.

Additionally, the system records detected objects along with timestamps, enabling monitoring and analysis of healthcare-related activities.

## 1.1 Scope of the Document

This document provides a comprehensive High-Level Design of the system, covering:

- System architecture
- Application workflow
- Information and process flow
- Component-level design
- Data handling mechanisms
- Interfaces and performance considerations

This document focuses on **overall system structure**, not detailed code implementation.

## 1.2 System Overview

The system performs **real-time object classification** using a pre-trained lightweight deep learning model.

### Key Functionalities

- Capture real-time video input
- Perform image preprocessing
- Run inference using TFLite model
- Display detected object labels

## Technologies Used

- Python
- TensorFlow Lite (TFLite)
- OpenCV
- NumPy

## 1.3 Intended Audience

- Students and Developers
- Project Evaluators and Examiners
- Technical Reviewers
- Academic Institutions

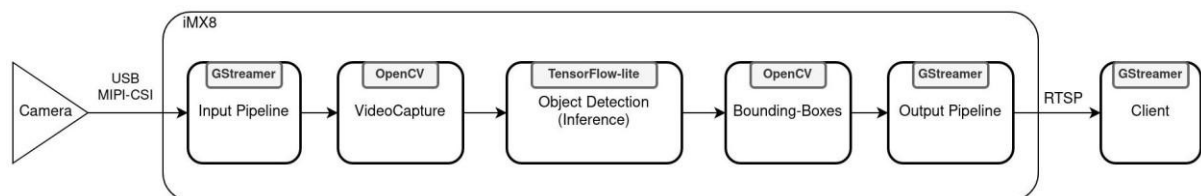
## 2. System Design

### 2.1 Application Design

The system follows a modular layered architecture:

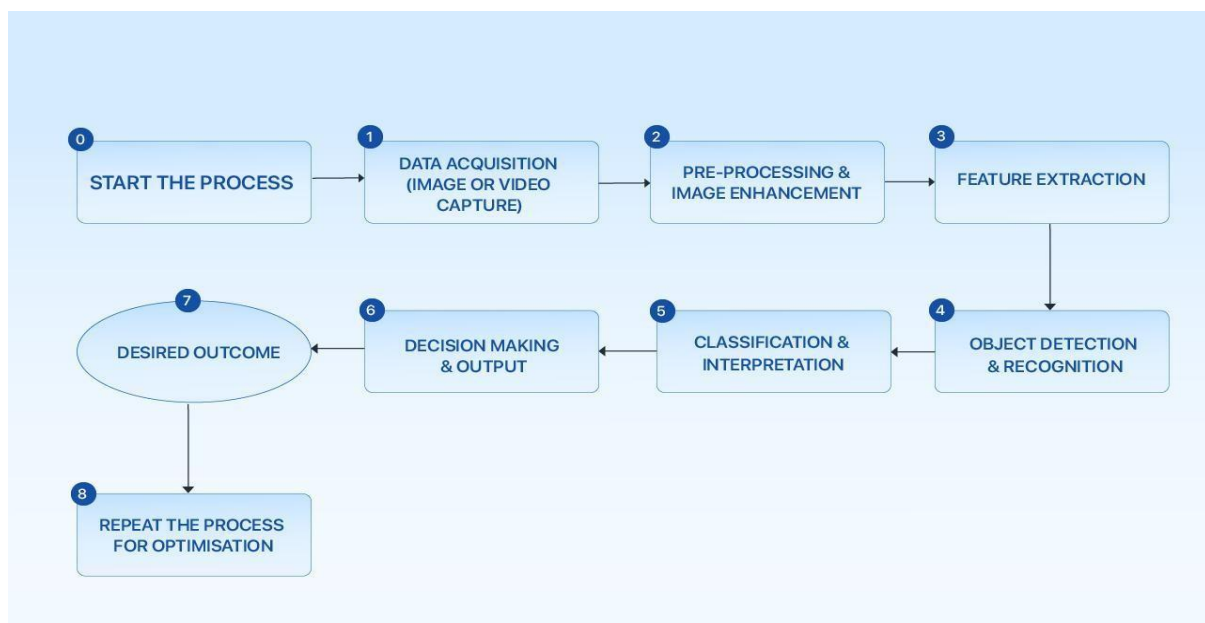
- **Input Layer** → Webcam input
- **Processing Layer** → Image preprocessing
- **Inference Layer** → TFLite MobileNet model
- **Output Layer** → Display and storage

### 2.2 Process Flow



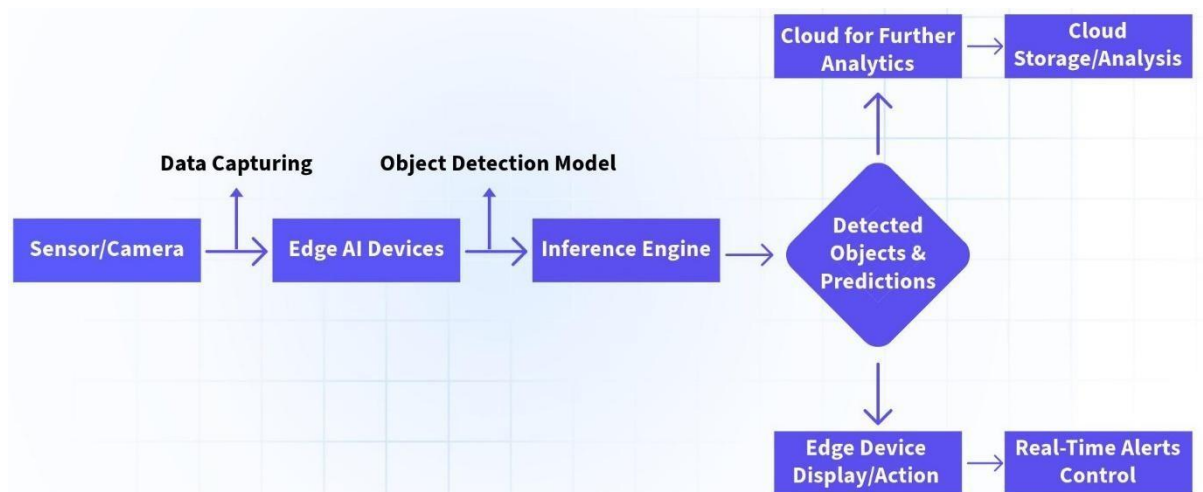
- Capture image frame from webcam
- Resize and normalize the image
- Feed processed image into TFLite model
- Perform inference (prediction)
- Extract class label and confidence score
- Display result on screen
- Store detection data in CSV file

## 2.3 Information Flow



- User starts the application
- Webcam captures continuous frames
- Frames are passed to preprocessing module
- Preprocessed data is fed into the inference engine
- Model generates predictions
- Output is displayed and stored locally

## 2.4 Components Design



- **Camera Module**  
Captures real-time video input using webcam
- **Preprocessing Module**  
Resizes, normalizes, and prepares image data
- **Inference Engine (TFLite Model)**  
Executes MobileNet model for prediction
- **Prediction Module**  
Extracts class label and confidence score
- **Display Module**  
Shows results on screen using OpenCV
- **Storage Module**  
Saves detection data (timestamp, label, confidence)

## 2.5 Key Design Considerations

- Lightweight model for edge deployment
- Real-time performance optimization
- Data privacy (no cloud dependency)
- Low computational resource usage
- Scalability for future enhancements

## 2.6 API Catalogue

| API Name                 | Description                 | Input       | Output          |
|--------------------------|-----------------------------|-------------|-----------------|
| <b>capture_frame()</b>   | <b>Captures image frame</b> | Webcam      | Image           |
| <b>preprocess()</b>      | <b>Processes image</b>      | Raw image   | Processed image |
| <b>run_inference ()</b>  | <b>Executes model</b>       | Image       | Prediction      |
| <b>display_output ()</b> | <b>Displays result</b>      | Label       | Screen output   |
| <b>save_data ()</b>      | <b>Stores detection</b>     | Label, Time | CSV entry       |

## 3.Data Design

### 3.1 Data Model

| Field Name        | Description           |
|-------------------|-----------------------|
| <b>Timestamp</b>  | Time of detection     |
| <b>Object</b>     | Detected object label |
| <b>Confidence</b> | Detection accuracy    |

### 3.2 Data Access Mechanism

- Data is stored in CSV format
- Accessed using Python file handling
- Append operation for real-time logging

### 3.3 Data Retention Policies

- Data stored locally on device
- No sensitive personal data involved
- Can be modified or deleted anytime

### 3.4 Data Migration

- Manual transfer of CSV files
- Future integration with databases (SQL/NoSQL)

## 4. Interfaces

- **User Interface**  
OpenCV display window
- **System Interface**  
Interaction between Python modules
- **External Interface**  
None (fully offline system)

## 5. State and Session Management

- Stateless architecture
- No user session required
- Each frame processed independently

## 6. Caching

- No caching implemented
- Future: caching prediction results



## 7. Non-Functional Requirements

### 7.1 Security Aspects

- Fully local processing
- No external data transmission
- Ensures data privacy

### 7.2 Performance Aspects

- Fast inference using TFLite
- Optimized CPU execution
- Real-time processing capability

## 8. References

- TensorFlow Lite Documentation
- OpenCV Documentation
- Python Official Documentation
- Edge AI Research Papers